

Volunteer shift **ELIGIBILITY**

Senior Software Engineer

Bloomerang Volunteer

A short, self-contained exercise built from a problem this team actually solves. Everything you need is in the folder. Nothing about nonprofits or volunteering is assumed.

How this works

- 1 Read the spec**
Open `SPEC.md`. It describes one question: can this volunteer sign up for this shift, and if not, why not? The sample data sits alongside it in `fixtures/`.
- 2 Build it**
Any language, any tools, any structure. Use whatever you normally reach for, including AI assistants—we do. There's no web service to stand up, no UI, and no database. A function and a way to run it is all we need.
- 3 Write down what you decided**
Fill in `DECISIONS.md`. This is the part we read most carefully, and it matters more than the code does.

About two to three hours.

It isn't timed. We don't know or record when you started, and there's no clock running. If you reach three hours and aren't finished, stop and write down what you'd do next. We'd rather read that than an exhausted submission—it tells us more about how you think.

What to send back

- **Your code and tests**, with a note saying how to run them.
- **Your completed `DECISIONS.md`**.
- **Your AI transcript or prompt log**, if you used one. We're not checking up on you. Directing a model well is a real skill, and we'd like to see how you do it.

Send it as a zip or a link to a repository—whichever is easier. If you used git, we'd enjoy seeing the history, but it isn't required.

One thing you should know

`SPEC.md` was written by a product manager working quickly, and it hasn't had an engineering review.

Part of the job is telling us what's wrong with it. Some of it is unclear, some of it doesn't say enough, and at least one part argues with itself. That isn't a trick—it's what most specs look like on the day they reach an engineer. We're interested in what you notice and, more than that, what you decide to do about it.

Building something different from what we'd have built costs you nothing, as long as you tell us why.

What we're looking for

Judgment

What you decided, what you deliberately left out, and why. Knowing what not to build is most of the job.

Clear thinking on paper

Could a product manager read your decision log and act on it? Is risk framed as consequence rather than mechanism?

Ownership

Tests that mean something, and an honest account of what's unfinished. We'd rather know than be impressed.

How you work with AI

Not whether you used it. How you directed it, what you verified, and anything it got wrong that you caught.

Working code matters, but it isn't what separates submissions.

What happens next

- 1 We read it**
Two engineers review your submission independently before comparing notes, so you get more than one read.
- 2 We talk it through**
A 45-minute conversation. You'll walk us through what you built and why. Expect us to ask about the parts of the spec you flagged, and about one you may not have.
- 3 We ask you to extend it**
We'll bring a change in requirements and talk through how you'd handle it. No preparation needed, and nothing to write in advance—it's a conversation, not a second exercise.

If you get stuck

If something is genuinely blocking you, email and ask. Noticing that a question needs asking is a good sign, not a bad one.

If you need an adjustment

If anything about this format doesn't work for you, tell us and we'll change it. That won't count against you in any way.

Good luck—we're looking forward to reading it.